Reinforcement Learning project : driving game

Matthieu DESTRADE, Pierre Emmanuel BAVIERES, Hugo AOYAGI, Raphaël MANUS

***Abstract*—This paper describes the application of Reinforcement Learning, with a specific focus on Deep Q-Learning and evolutionnary algorithms, to navigate a driving game where the player steers a car on a randomly generated road. Our goal was to enable our model to engage with the game solely based on the information accessible to a human player, i.e., the visual display of the game. Models of this nature hold potential applications in robotics and even self-driving technology. Within this report, we will detail the methods employed and the selected architecture to optimize our model for achieving the most effective agent in our game.**

I. Introduction

Our primary intention for this project was to ensure that our model had access to the same variables and information about the environment as a human would do. Consequently, training a model to choose an action based on the distance between the car and the center of the road was ruled out, as a human would not be able to access this value directly while playing the game. Therefore, we needed to provide our model with the image of the game at each frame. This approach necessitates the use of deep Q-Learning, and in our case, deep convolutional Q-Learning to extract crucial information from the images. Additionally, we implemented various reinforcement learning (RL) methods to achieve optimal performance, such as frame-skipping and buffers.

Since deep Convolutional Q-Learning requires no information beyond what the user sees, it presents a highly interesting approach applicable to a wide variety of games or other applications, such as robotics. Moreover, as applications grow in complexity, there is a demand for scalable reinforcement learning (RL) models, a need addressed by Deep Q-Learning. The architecture of the model can be tuned to adapt to the specific requirements of diverse applications.

We encountered various challenges throughout the project, with one of the initial obstacles revolving around computational complexity. Specifically, presenting the same image to the model as a human agent and selecting an action for each frame would necessitate extensive computations due to the model's substantial number of parameters. This issue was addressed by providing the model with a compressed version of the image that retained essential information. In a broader context, careful consideration was given to the images fed into the network. Initially, we aimed to simplify them to evaluate the capabilities of our method. Additionally, we grappled with challenges related to choosing the reward function. We aimed to strike a balance between not providing excessive information to the agent while still guiding it toward the desired behavior. Finally, one of our most significant challenges was the management of time. Given that the agent is expected to play in real time, we had to prioritize the time difference between two frames of the game and the duration for which an action chosen by the model should be applied.

Convolutional Deep Q-Learning has already been employed to develop agents for different games, including Doom. In our class, we observed that Deep Q-Learning has a few limitations, such as high data correlation and the challenge of dealing with a constantly moving target—in our case, the road randomly changes continuously. Both of these issues can be addressed through the implementation of a replay buffer and a target network, as we will elaborate on later in this report.

We therefore developed a simple one-player game in which the user can drive a car and must navigate a randomly generated road. The car's speed is higher when it stays on the road, and it has limited mobility outside of it. Subsequently, we transformed this game into a `gymnasium` environment. Finally, we experimented with two reinforcement learning techniques to train an agent to perform as well as or better than a human. We utilized the deep Q-learning implementation from the course and the `evotorch` library for evolutionary algorithms.

We successfully trained an effective agent using the first method (deep Q-learning) that can stay on the road for an extended period. The agent also demonstrated a sense of path optimization, as it cut some curves in the road. Our method proved its efficiency, yielding the desired model in a relatively short computation time, approximately 1 hour.

Our agent still has some limitations. It eventually loses stability and leaves the road. Additionally, its behavior is not very smooth; it tends to change direction frequently, even when it is not necessary. Therefore, it could be interesting to try fine-tuning it by adjusting various parameters. For example, a modified reward function or experimenting with different types of roads might be beneficial. One approach could involve training an already well-performing agent on more complex roads than the ones encountered in the base game. Other avenues we could explore are linked to the simplification of the observations provided to the model. We could attempt to utilize more complex observations and assess whether we can still train an efficient agent within a reasonable timeframe with them.

Our code is available at https://github.com/matthieuDESTRADE/CarRL . The file `DQN_main.ipynb` can be used to train a new model, and its last cell can be used to run the game with a pretrained model. The file `evolutionary_algos.ipynb` can be used to do the same for genetic approaches.

II. Background

Reinforcement learning (RL) provides a framework for learning optimal decision-making through interactions within

an environment, framed within a Markov Decision Process (MDP). This mathematical framework consists of states (s), actions (a), rewards (r), and policies (π_θ) parameterized by θ. A policy dictates the agent's action in a given state, aiming to maximize cumulative rewards over time. A stochastic policy is actually of distribution of probability among the action space. The trajectories ($\boldsymbol{\tau}$) that an agent takes through the state space are influenced by a probability distribution p_θ, shaped by the policy and the environment's dynamics [1].

Our project is inspired by the integration of deep learning with RL. This enables the handling of high-dimensional sensory inputs, such as those from video frames in a racing game, allowing for the extraction of features directly from the input data without manual feature engineering [3]. Indeed, neural networks, and in particular convolutional neural networks (CNN), have been proven very efficient in computer vision. State of the art algorithms are able to make object classification in real time on video frames [10]. CNNs allow to design policy that can handle complex input and then play accordingly.

Deep Q-Networks (DQN) combine Q-learning, a value-based method for finding the optimal action-selection policy, with the high-capacity function approximation of deep neural networks. The principle of Q-learning consists in estimating the future rewards for the whole game as a consequence of each possible action. The DQN's architecture, particularly its use of a replay buffer to store and reuse past experiences, mitigates the correlations between consecutive observations, stabilizing the learning updates [3].

Evolutionary algorithms (EA) are population-based algorithms trying to select the most adapted individual to a given task. Genetic algorithms offer a novel approach to exploring the policy space, optimizing the agent's behavior through mechanisms inspired by natural evolution, such as selection, crossover, and mutation. This diversity in training methodologies enhances the robustness and adaptability of the learned policies [6][7]. This set of algorithm can be used along CNN-based policies. In such case, each individual consists in all the weights of the neural network. Many different algorithmic choice can then be made to select the best individuals. Some EAs are considered as black-box optimizers. With very limited knowledge on the problem to solve they can provide very good results.

The game's environment is a 2D racing game developed with Pygame, featuring a track generated through Perlin noise—a technique for producing smooth random functions. This procedural generation method ensures each session presents a unique path, requiring the RL agent to continually adapt its strategy to navigate the track successfully. The use of Perlin noise for track generation is crucial for creating varied and unpredictable game scenarios, challenging the agent's ability to generalize and learn from diverse experiences [4].

In controlling the vehicle, the agent must manage acceleration, braking, and steering actions, reflecting the complexities of real-world driving. This dynamic control task is formulated as an RL problem, where the agent learns to optimize its control policy over time through trial and error, guided by a reward function. The reward function is designed to encourage the agent to stay on the track and maintain high speed, incorporating penalties for going off-track, thus aligning the agent's objectives with the game's goals [5].

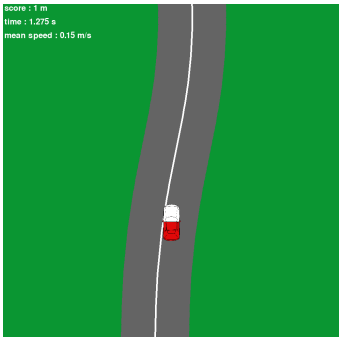

Fig. 1. A frame of our racing game

The model's architecture, tailored for processing game frames, is a critical element in the agent's ability to perceive and interact with the game environment effectively. This involves convolutional layers for feature extraction from raw pixel data, followed by fully connected layers for action value estimation. Such a design is instrumental in enabling the agent to make informed decisions based on its visual perception of the game scenario, simulating the cognitive processes involved in human game-playing [8]. The choice of the type and the size of each layer is also crucial. A too simple model may not be able to make competitive decisions. On the other hand, a too large model may be untrainable for computer complexity reasons. We adopted a classic model that we found from multiple papers such as [9] and [11] when it comes to basic feature extraction.

III. Methodology/Approach

A. Deep Q-learning

To create an agent capable of playing our game, we established an environment using the `gymnasium` framework, enabling the application of traditional Reinforcement Learning (RL) algorithms. The various components necessary for building the environment were defined as follows:

- **Action space**: Our action space is straightforward, encompassing the fundamental inputs a human player would employ to navigate the game—specifically, eight actions: up, down, left, right, up and left, up and right, down and left, and down and right. The action space is binary; the agent lacks the ability to precisely determine the degree of leftward turn. To account for this limitation, the agent must repeat the same action over multiple frames.
- **Step function and initialization of the environment**: We adapted our game code to initialize the environment and calculate the subsequent frame based on the chosen action. When initializing the environment for training, we rotate the car by a random angle, so that it cannot only go forward to gain rewards.
- **Reward function**: We selected a reward function that yields the distance traveled on the road, augmented by a small component proportional to the distance traveled toward the center of the road. These distances were

computed using scalar and cross products between the speed vector of the car and a vector collinear with the direction of the road at its closest point to the car. This design encourages the car to maintain a position in the middle of the road. With the same objective in mind, we opted to trigger the `done` flag as soon as the car deviates from the road, thus reinitializing the environment. Additionally, this flag is activated after a number of frames equivalent to 30 seconds at 30 frames per second have been processed. We observed that suppressing the strategies employed to align the car with the center of the road can fine-tune and enhance the performance of an already proficient agent.

- **Observation function**: To train an agent relying solely on the game's visual information, our observation function returns a downsized (40×40) colored version of the frame visible to the player. For computational efficiency, we opted to streamline the car's design and omitted certain visual elements, retaining only a dark line for the road, a green background, and a white and red car.

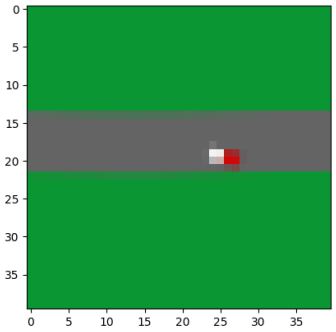

Fig. 2. An example of an observation given to the model

To train the agent, we primarily explored two methods. Initially, we experimented with a deep Q-Network incorporating infrequent weight updates. To this end, we created :

- **A Q-network**: With the aim of deriving Q values from a frame, we employed a deep convolutional neural network comprising three layers with max-pooling and leaky ReLU, followed by two feedforward layers.
- **An epsilon-greedy strategy**, returning the best action chosen by the Q-network with a probability of 1-ϵ, and otherwise, a random action chosen with a probability of ϵ (which decays over time).
- **A replay buffer**, which we imported from the `torchrl` library, allowing us to easily save observations, decisions and rewards in order to reuse them to train our model.
- **A train function**: We employed the DQN with infrequent weight updates algorithm, utilizing two instances of the Q-network. One instance serves as a target and is only updated after a specified number of iterations of the algorithm, and the other is trained at every step using batches of data from the replay buffer. Each epoch corresponds to one attempt by the agent. To address a challenge observed in the model's ability to comprehend the effects of its actions when applied for only one frame, we implemented a mechanism to select the number of frames on which a chosen action would be executed. This trick is known as frame-skipping, and skipping several frames allows us to give more weight to the decisions of the model, which is very useful at least for the beginning of the training.

After several trials, we discovered a highly efficient approach to training the model with these tools. Initially, we train the algorithm while applying each action over a sequence of 10 frames. Subsequently, we fine-tune the model obtained while applying each action to a single frame. The training curves are available in the appendix.

B. Evolutionary algorithms

We then decided to investigate if classic evolutionary algorithms could give promising results as well. Instead of implementing from scratch some evolutionary algorithms, we leveraged the genericity of the `gymnasium` framework by using the `evotorch` library. `evotorch` is a python interface that instantiate highly efficient compiled objects. It is compatible with `torch` tensors and implements several evolutionary algorithms.

The purpose of EAs is too optimize a function called an objective whose parameters are the parameters of the policy. In the case of reinforcement learning, a classic objective function is to take the cumulative reward (or average reward depending on the problem) through one (or several) episode played with a given policy. The `GymNE` interface is implementing exactly this approach. The policy is a neural network, evaluated on observation to get the next action to play. The cumulative reward of the episode is used as an estimation of the performance of the NN.

The first step of the method is to evaluate every individual (i.e. every policy) of the samples with the objective function. Then we determine which will be the new individuals of the population based on the fitness of the previous. This is the key part of each algorithm. There are plenty of ways to do so. The assumption is that the best individual will be similar to the ones performing well on the current iteration.

To this end we mainly used Gaussian search algorithms. A gaussian search algorithm uses a multivariate gaussian law as a search distribution to sample the children. This method is called parametric because the distribution admit the parameter θ containing the centers μ and the covariance matrix σ of each gaussian. The NES algorithm follows the gradient of the fitness function to update of the search distribution. The SNES assumes that it is sufficiant to consider diagonal covariance matrices. This means that no coordinate is priviledged. This search distribution is simpler and decrease the computational cost. Finally, the CEM algorithm iteratively updates θ to maximize the expectation of the log likelihood function of θ.

However, at first, the algorithms CEM and SNES all ended in a local minimum where the car only went straight forward. Indeed, this is a plausible easy solution to get a huge amount of rewards without taking any risk. To work around this issue, we considered using an alive bonus. An alive bonus is added to the rewards for the time stayed alived. Alike using a high

γ in the Q-value function, this puts the focus on the future reward and allow for longer runs. The possible drawback is to have carefull survivors that only accumulate this alive bonus while staying extremely safe.

IV. Results

We managed to get very promising results with the deep Q-learning method. Here are the results of several models we trained using our method, evaluated using the following procedure: we simulate 10 tries (limited to 30 seconds) in the game using the model and return the average score.

TABLE I
Results Deep Q-learning

Model	Average score
random	-1.10
only accelerate	74.85
DQL	82.07
FT-DQL	168.87
VG	**307.78**
VG2	255.62

The descriptions of the models are the following:

- **DQL** : trained for 300 episodes with a frame-skipping of 10.
- **FT-DQL** : fine tuned from DQL for 300 episodes successively with frame-skippings of 50, 10, 5 and 1.
- **VG** : Uses a smaller action space (no left, right, bottom-left, bottom-right). Trained for 300 episodes with a frame-skipping of 10, then fine-tuned for 300 episodes without a frame-skipping.
- **VG2** : fine tuned from VG with a score that does not use the distance to the center of the road (300 episodes, no frame-skipping).

Our final best agent (VG) is able to stay on the road for a very long time, and even to go back to the road if it leaves it. The fact that VG models are better than DQL ones implies that some actions are useless for good performances and only divert the model from the optimal strategy. In all cases, there is still room for improvement as sometimes the agent gets unstable and leaves the road indefinitely. Moreover, while it sometimes shows signs of path optimization, the agent tends to change direction frequently and 'vibrate.' Fine-tuning could likely smooth its trajectory

Additionally, we think that we could use the actual training as a starting point to play a more complex game. In our case, the speed limit is quickly reached and the agent only has to go left or right and push forward. By increasing this speed limit and introducing drift, the car would go out of the road more often and would have to learn to drive more carefully.

However, concerning the evolutionary methods, we were not able to get convincing results:

Using these methods, we only got agents that always went forward.

To further improve our results, we considered using contributions from computer vision work to speed up the training. By applying basic line recognition we could easily downsample the input space. From a 40×40 RGB frame we could end with only the knowledge of the car with respect to the road (oriented vector from the closest center of the road) and its orientation. This information would be sufficient to train an agent and make steering decision. We could also use a proportional–integral–derivative controller to mitigate overshooting and error accumulation. Another idea is to use neural network already trained for computer vision. This method called transfer-learning makes use of the feature extraction to foster training on the race game specific problem. It has been used in [9]. Obviously, these technique would go against the idea we developped of giving the agent only the frame and no further precomputation. This precomputation would have made the problem simpler but also more specific to the game we are playing with a single road. On the contrary, Deep-Q-learning and EAs can generalize better if using several lanes or other cars.

TABLE II
Results evolutionary methods

Model	Average score
random	-1.10
only accelerate	74.85
CEM	93.82
CMAES	80.13
SNES	82.90

To gain a deeper understanding of the model's inner workings, we were finally keen on exploring how the various layers interpret the game images we input into the model. To accomplish this, we defined a function that calculates the mean activation of a channel in a convolutional layer. By identifying images that maximize this function, we can observe the patterns recognized by the layer. This is achieved by employing the Adam Optimizer on random images, visualizing the resulting image after 200 optimization steps, and using the VG2 model.

In the following image, we applied our method to the second convolutional layer on its fifth channel. We observe that this channel is activated by changes from red to white. Furthermore, when we input an actual game image through this channel, we actually see that this channel is activated in the regions where the car is present. This is coherent as the car is two-colored with its front white and back red.

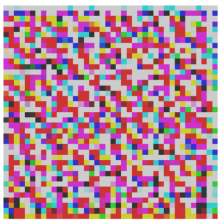

Fig. 3. Maximum activation

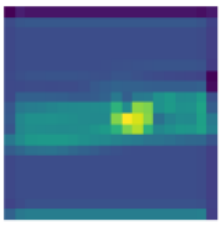

Fig. 4. Activation of layer from game image

We can observe the same with some channels recognizing patterns with black and green lines, these channels actually recognize the edges of the road.

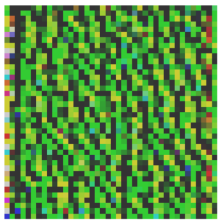

Fig. 5. Image maximizing activation

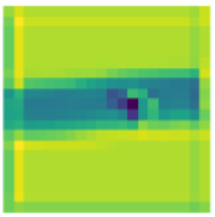

Fig. 6. Activation of layer from game image

This analysis highlights the patterns that allow the model to find the posititon of the car and the position of the road around it, and shows that our model has effectively learnt how to use the principal characteristics of a game image.

V. Conclusions

To conclude, we successfully developed a driving game, implemented it in a `gymnasium` environment, and employed deep Q-learning and evolutionary algorithms to train an agent capable of playing the game based solely on frames. Throughout this project, we gained valuable insights into the significance of techniques like frame-skipping for efficient model training. Additionally, we had the opportunity to observe how a convolutional network extracts information from an image. While our results are promising, further improvement could be achieved through fine-tuning the model and training the agent in new and more challenging scenarios.

References

[1] Sutton and Barto. Reinforcement Learning, *MIT Press*, 2020.

[2] Read. Lecture IV - Reinforcement Learning I. In *INF581 Advanced Machine Learning and Autonomous Agents*, 2024.

[3] Mnih, V., et al. "Playing Atari with Deep Reinforcement Learning." *arXiv:1312.5602*, 2013.

[4] Perlin, K. "An Image Synthesizer." In *SIGGRAPH Comput. Graph.*, 19(3), 287-296, 1985.

[5] Watkins, C.J.C.H. Learning from Delayed Rewards. *PhD Thesis, University of Cambridge, England*, 1989.

[6] Lin, L.-J. "Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching." In *Machine Learning*, 8(3-4): 293-321, 1992.

[7] Holland, J.H. Adaptation in Natural and Artificial Systems. *University of Michigan Press*, 1992.

[8] LeCun, Y., Bengio, Y., & Hinton, G. "Deep Learning." In *Nature*, 521(7553), 436-444, 2015.

[9] Aldape and Sofwell. Reinforcement Learning for a Simple Racing Game. (2018). Stanford University.

[10] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once : Unified, Real-Time Object Detection. Arxiv.

[11] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot and F. Nashashibi, "End-to-End Race Driving with Deep Reinforcement Learning," 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 2018, pp. 2070-2075, doi: 10.1109/ICRA.2018.8460934.

VI. Appendix

Here are examples of the performances of the agents throughout training.

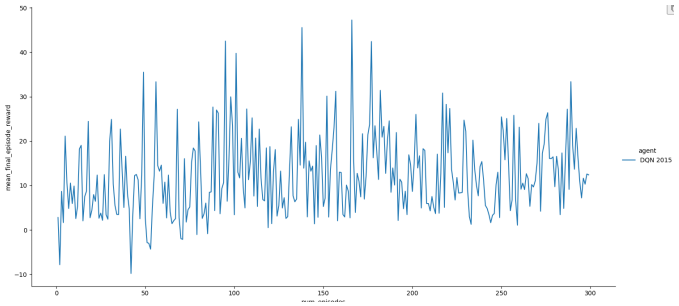

Fig. 7. Training curve with frame-skipping equal to 10

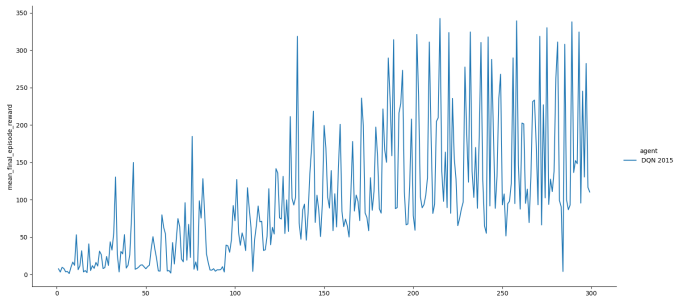

Fig. 8. Training curve for the fine-tuning of the previous model with frame-skipping equal to 1